Snowflake Hands-on Documentation

IPL Dataset • Setup, CSV loading, verification and cleanup

Snowflake objects — quick beginner notes

Database

- A database is the top-level container that organizes related data objects in Snowflake.
- We created **IPL_DB** to keep the IPL schema and table together.

Schema

- A schema is a logical folder inside a database that groups tables, stages and file formats.
- We used **IPL** to keep all IPL-related objects organized within IPL_DB.

Table

- A table stores the actual structured data in rows and columns.
- We created **IPL_DATA** to store the IPL match records loaded from the CSV.

File Format

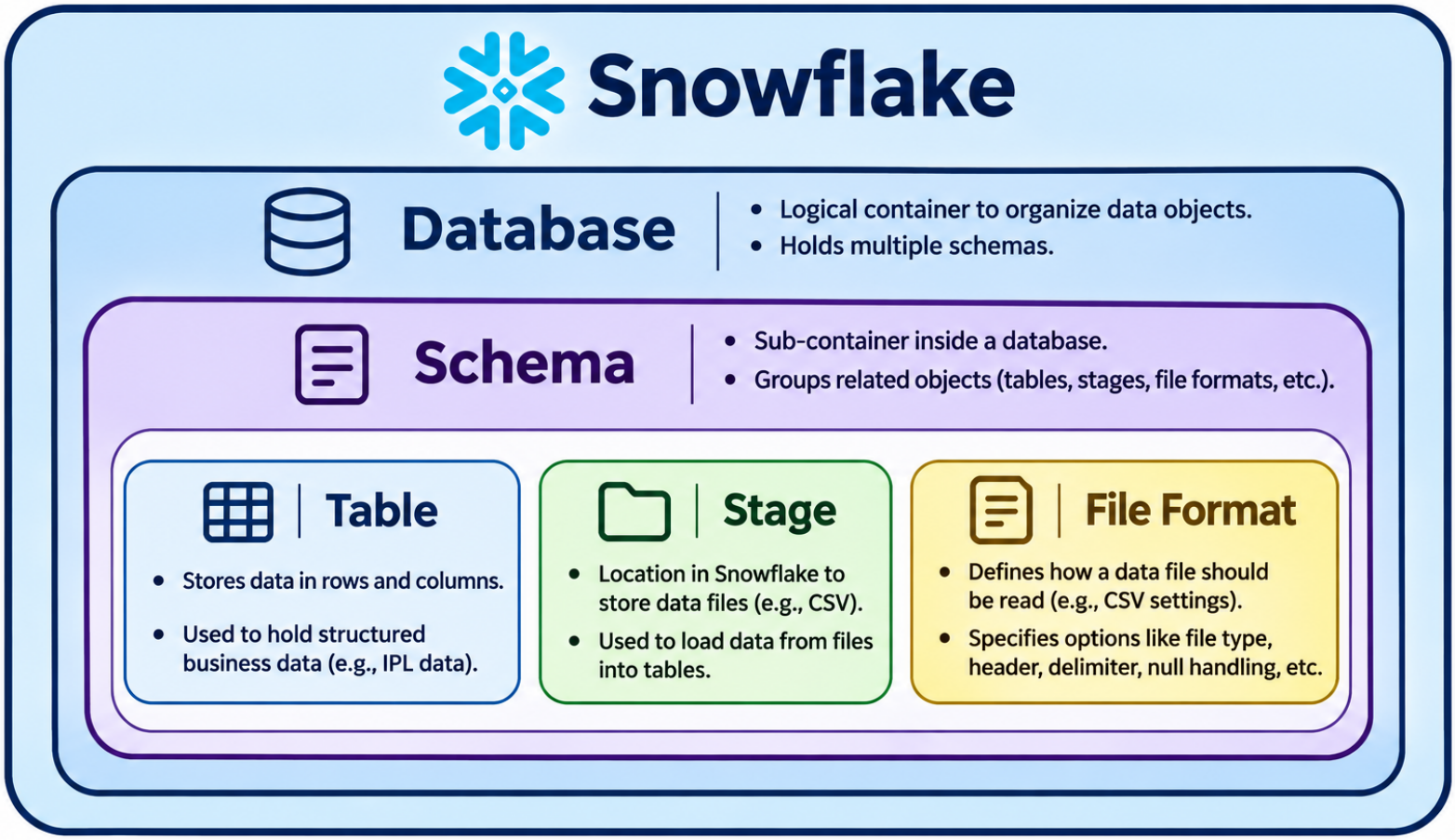
- A file format tells Snowflake how to read a file, such as CSV, including headers, delimiters and quotation marks.
- We created **IPL_CSV_FORMAT** so Snowflake could correctly interpret the uploaded CSV.

Stage

- A stage is a storage location that holds files before Snowflake loads them into a table (or after data is exported).
- We used the internal stage **IPL_STAGE** as the landing area for ipl_data.csv.

COPY INTO

- **COPY INTO** is the Snowflake command used to load data from staged files into a table.
- It connects the stage, file format and destination table; here it loaded the CSV rows into IPL_DATA.


Snowflake
Database
Logical container to organize data objects.
Holds multiple schemas.
Schema
Sub-container inside a database.
Groups related objects (tables, stages, file formats, etc.).
Table
Stores data in rows and columns.
Used to hold structured business data (e.g., IPL data).
Stage
Location in Snowflake to store data files (e.g., CSV).
Used to load data from files into tables.
File Format
Defines how a data file should be read (e.g., CSV settings).
Specifies options like file type, header, delimiter, null handling, etc.

Hands-on SQL steps

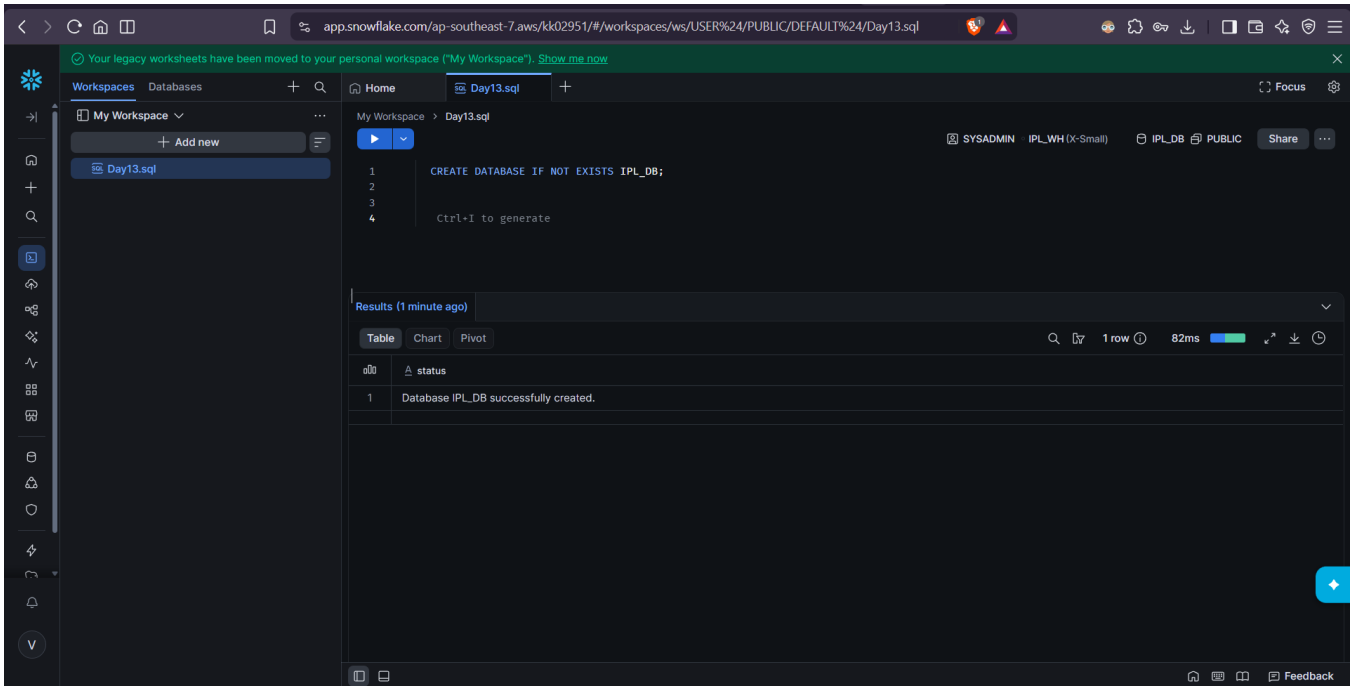
Step 1: Create IPL Database

SQL Command

```
CREATE DATABASE IF NOT EXISTS IPL_DB;
```

Output

Database IPL_DB successfully created.



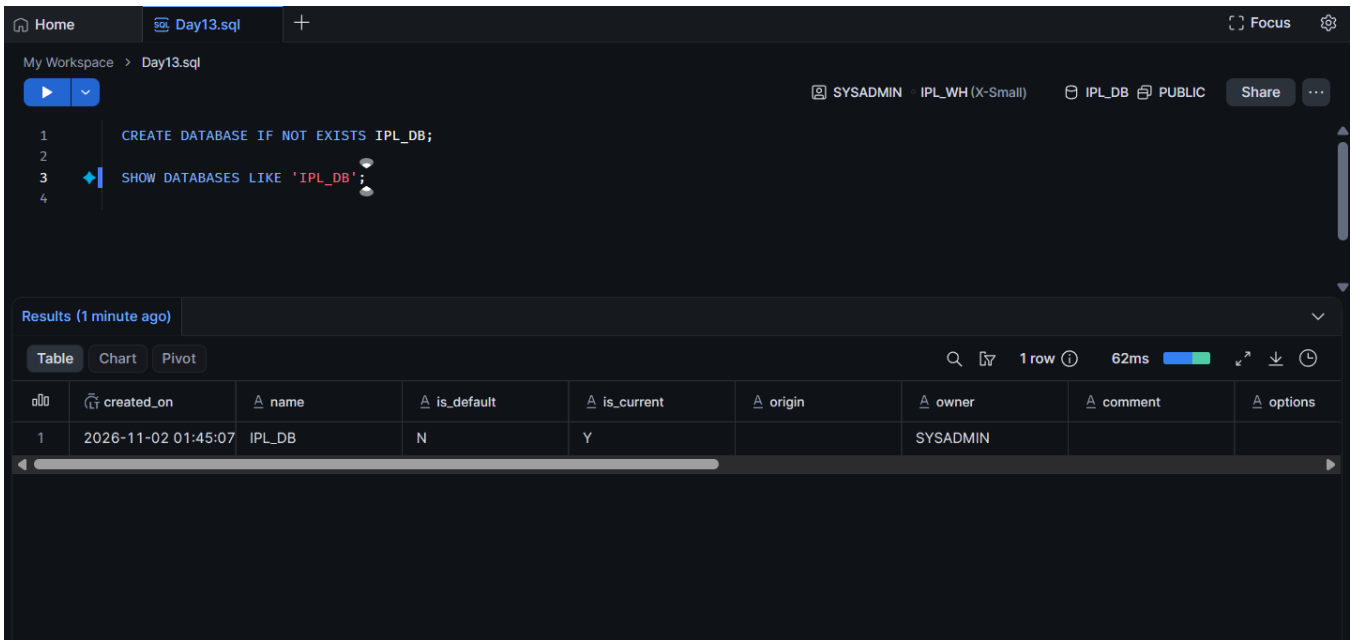
Step 2: Verify Database Creation

SQL Command

```
SHOW DATABASES LIKE 'IPL_DB';
```

Output

IPL_DB is listed in the results.



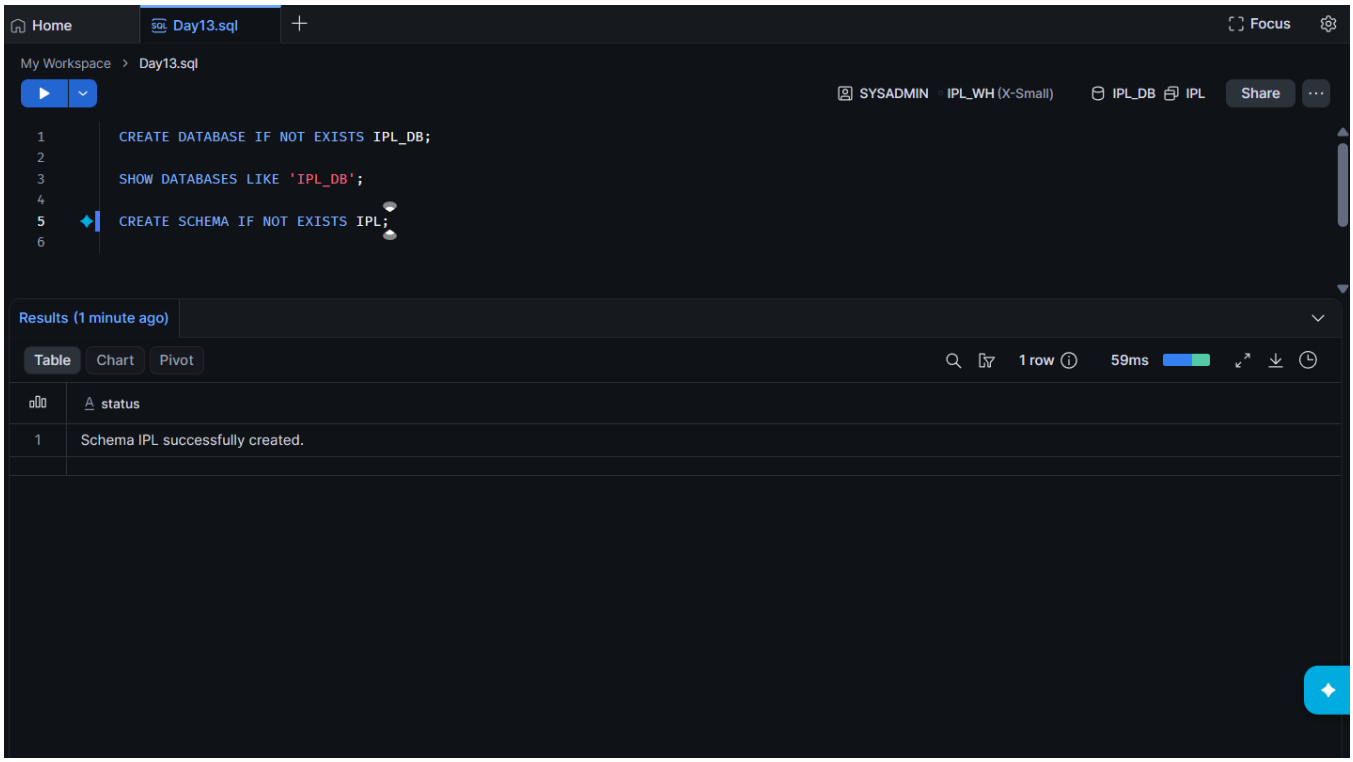
Step 3: Create IPL Schema

SOL Command

```
CREATE SCHEMA IF NOT EXISTS IPL;
```

Output

Schema IPL successfully created.



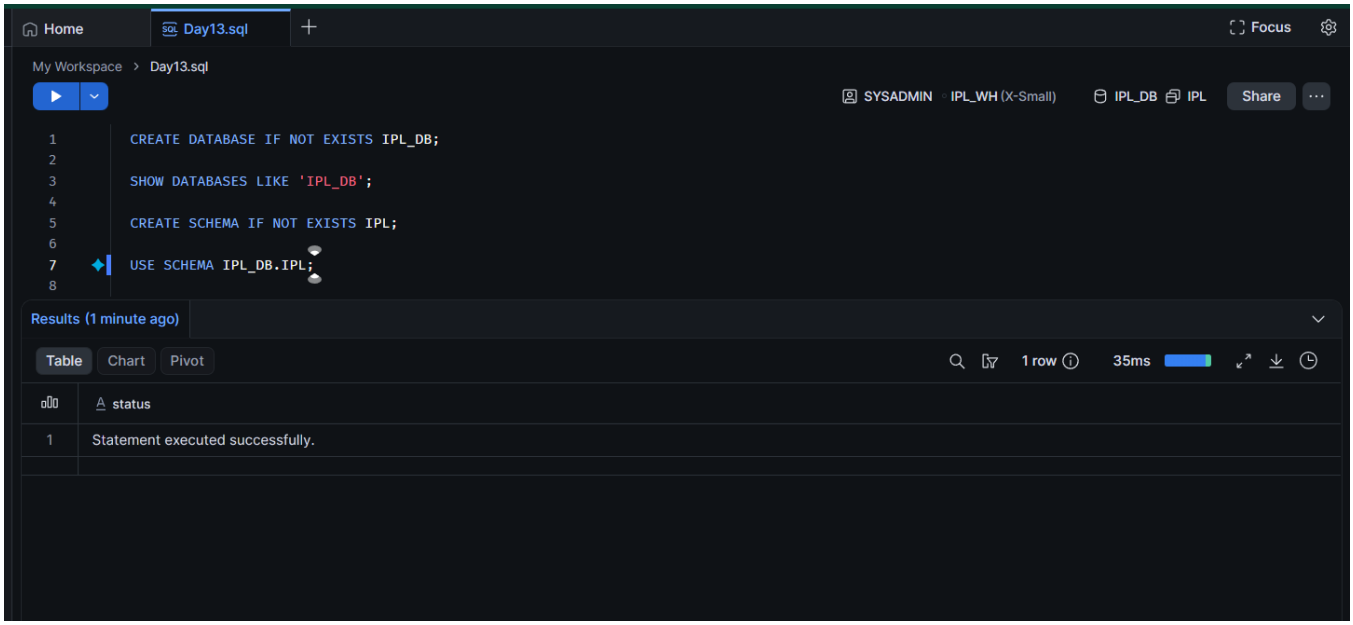
Step 4: Select IPL Schema

SOL Command

```
USE SCHEMA IPL_DB.IPL;
```

Output

Statement executed successfully.



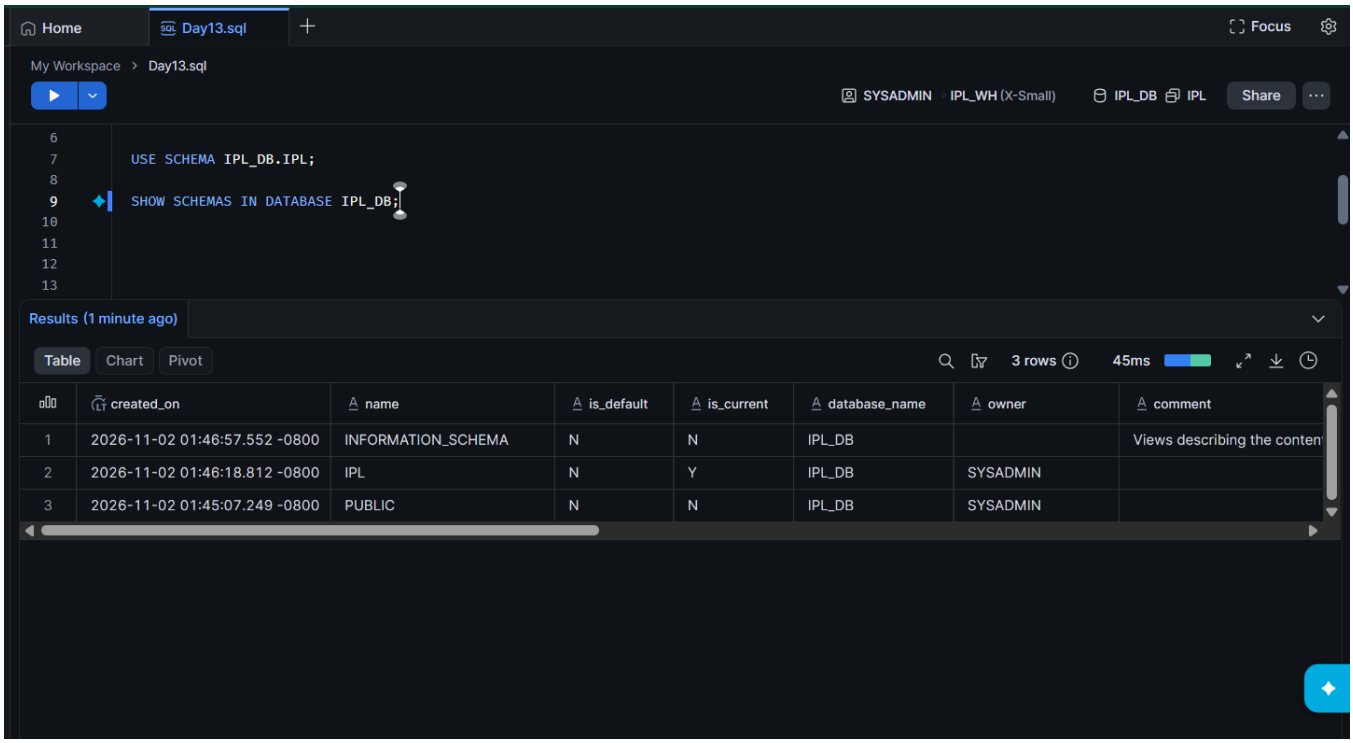
Step 5: Verify Schemas

SOL Command

```
SHOW SCHEMAS IN DATABASE IPL_DB;
```

Output

The results list INFORMATION_SCHEMA, IPL and PUBLIC.



	created_on	name	is_default	is_current	database_name	owner	comment
1	2026-11-02 01:46:57.552 -0800	INFORMATION_SCHEMA	N	N	IPL_DB		Views describing the conten
2	2026-11-02 01:46:18.812 -0800	IPL	N	Y	IPL_DB	SYSADMIN	
3	2026-11-02 01:45:07.249 -0800	PUBLIC	N	N	IPL_DB	SYSADMIN	

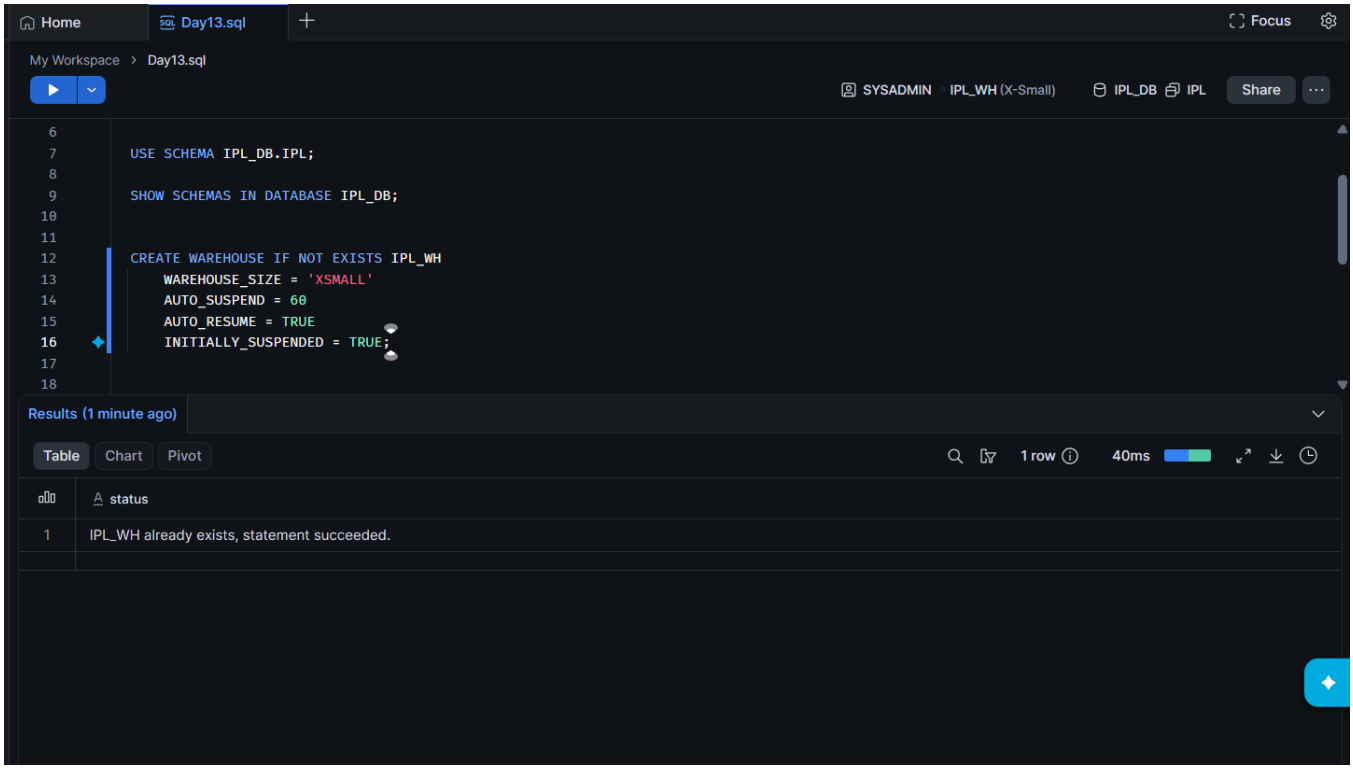
Step 6: Create Virtual Warehouse

SOL Command

```
CREATE WAREHOUSE IF NOT EXISTS IPL_WH
WAREHOUSE_SIZE = 'XSMALL'
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE
INITIALLY_SUSPENDED = TRUE;
```

Output

IPL_WH already exists; statement succeeded.



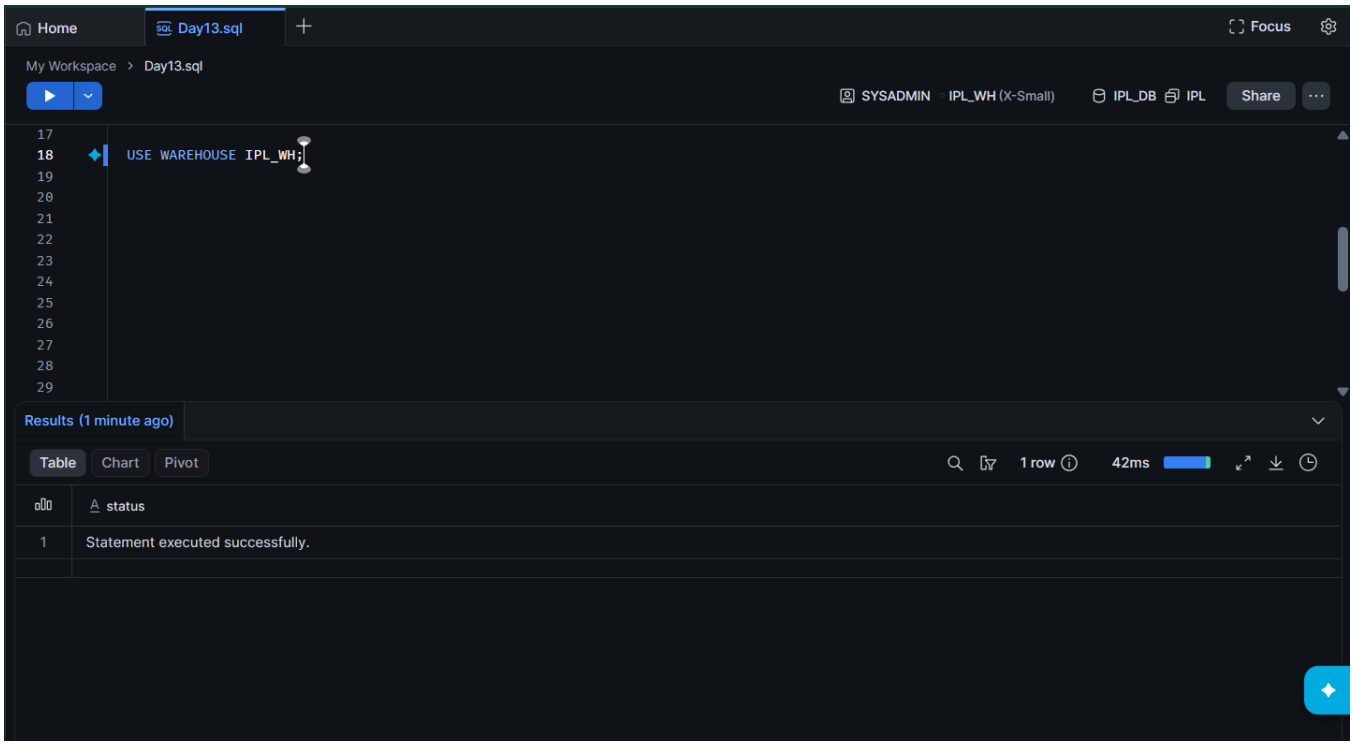
Step 7: Select Virtual Warehouse

SOL Command

```
USE WAREHOUSE IPL_WH;
```

Output

Statement executed successfully.



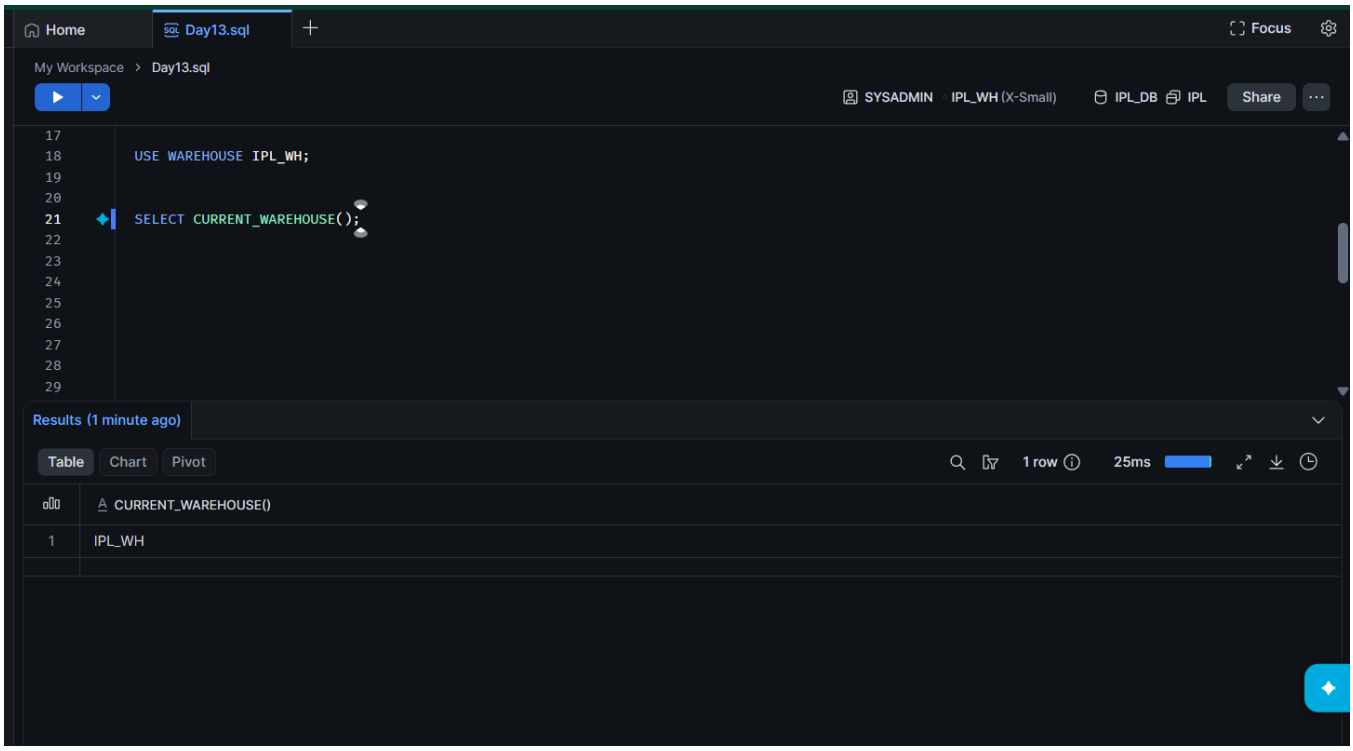
Step 8: Verify Current Warehouse

SOL Command

```
SELECT CURRENT_WAREHOUSE();
```

Output

The current warehouse is IPL_WH.



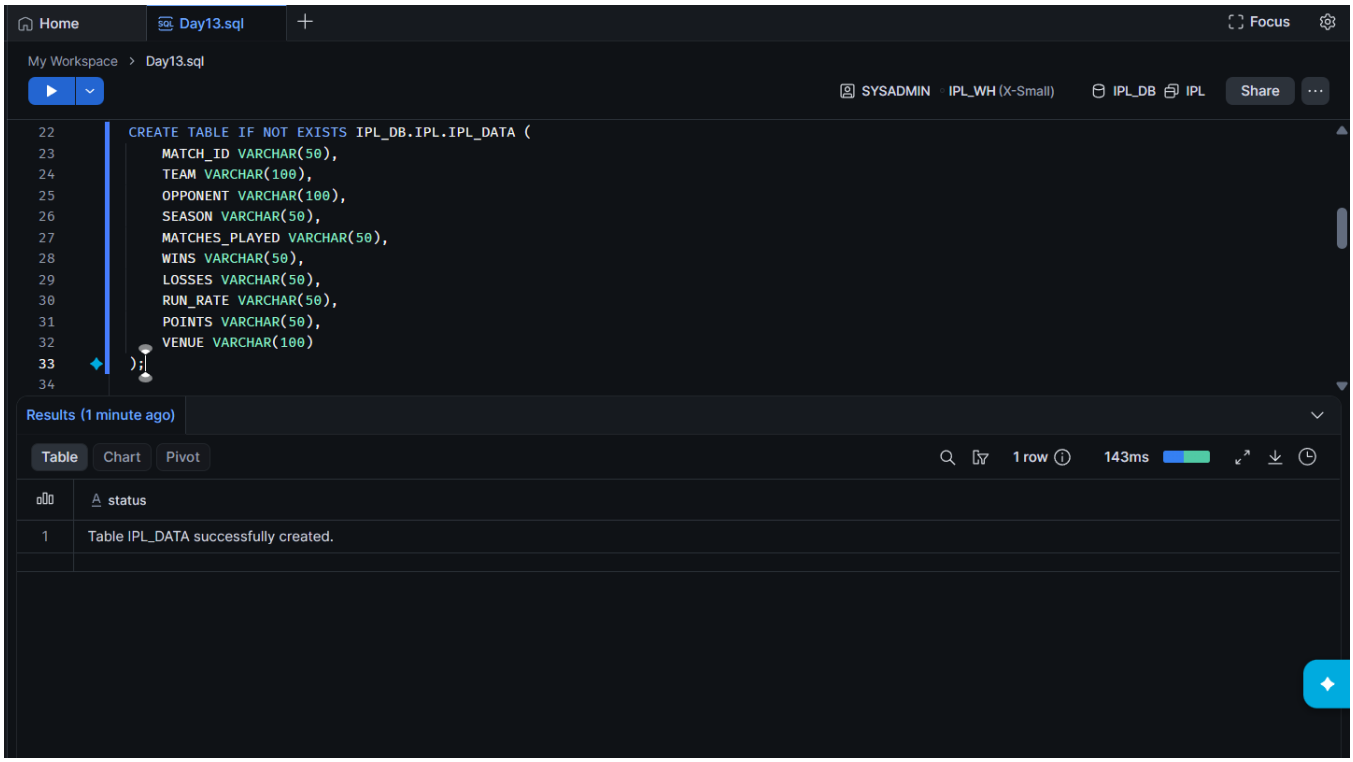
Step 9: Create IPL Data Table

SOL Command

```
CREATE TABLE IF NOT EXISTS IPL_DB.IPL.IPL_DATA (
MATCH_ID VARCHAR(50),
TEAM VARCHAR(100),
OPPONENT VARCHAR(100),
SEASON VARCHAR(50),
MATCHES_PLAYED VARCHAR(50),
WINS VARCHAR(50),
LOSSES VARCHAR(50),
RUN_RATE VARCHAR(50),
POINTS VARCHAR(50),
VENUE VARCHAR(100)
);
```

Output

Table IPL_DATA successfully created.



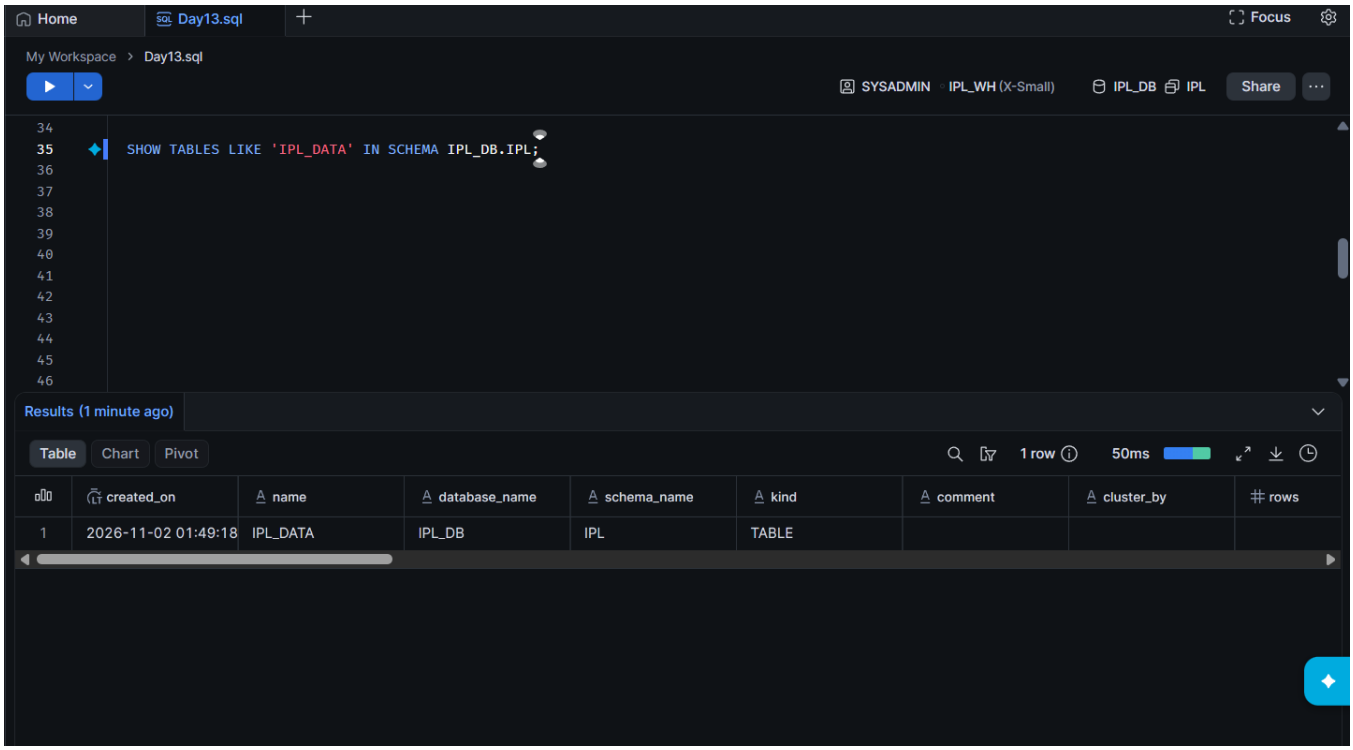
Step 10: Verify Table Creation

SOL Command

```
SHOW TABLES LIKE 'IPL_DATA' IN SCHEMA IPL_DB.IPL;
```

Output

IPL_DATA is listed as a TABLE in schema IPL.



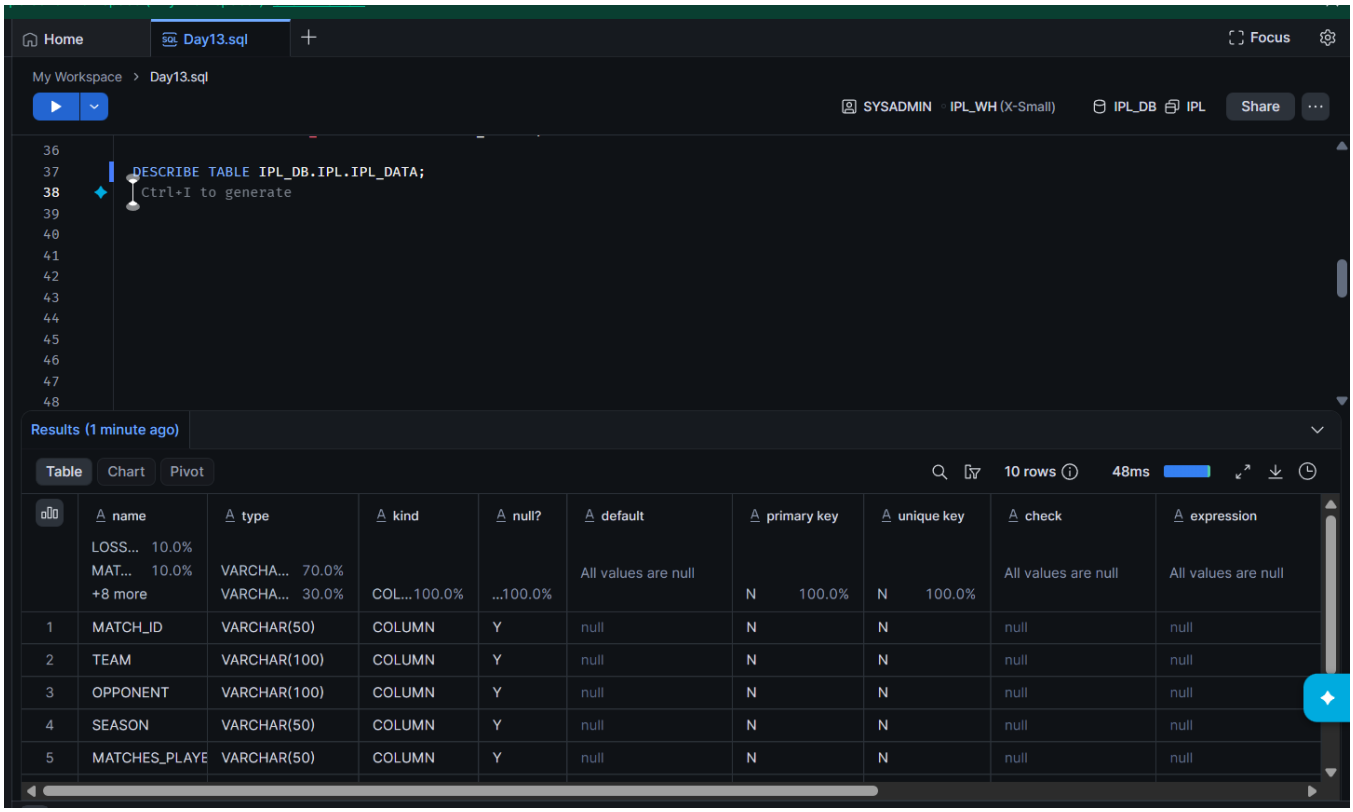
Step 11: Describe IPL Data Table

SOL Command

```
DESCRIBE TABLE IPL_DB.IPL.IPL_DATA;
```

Output

The table description displays 10 columns and their data types.



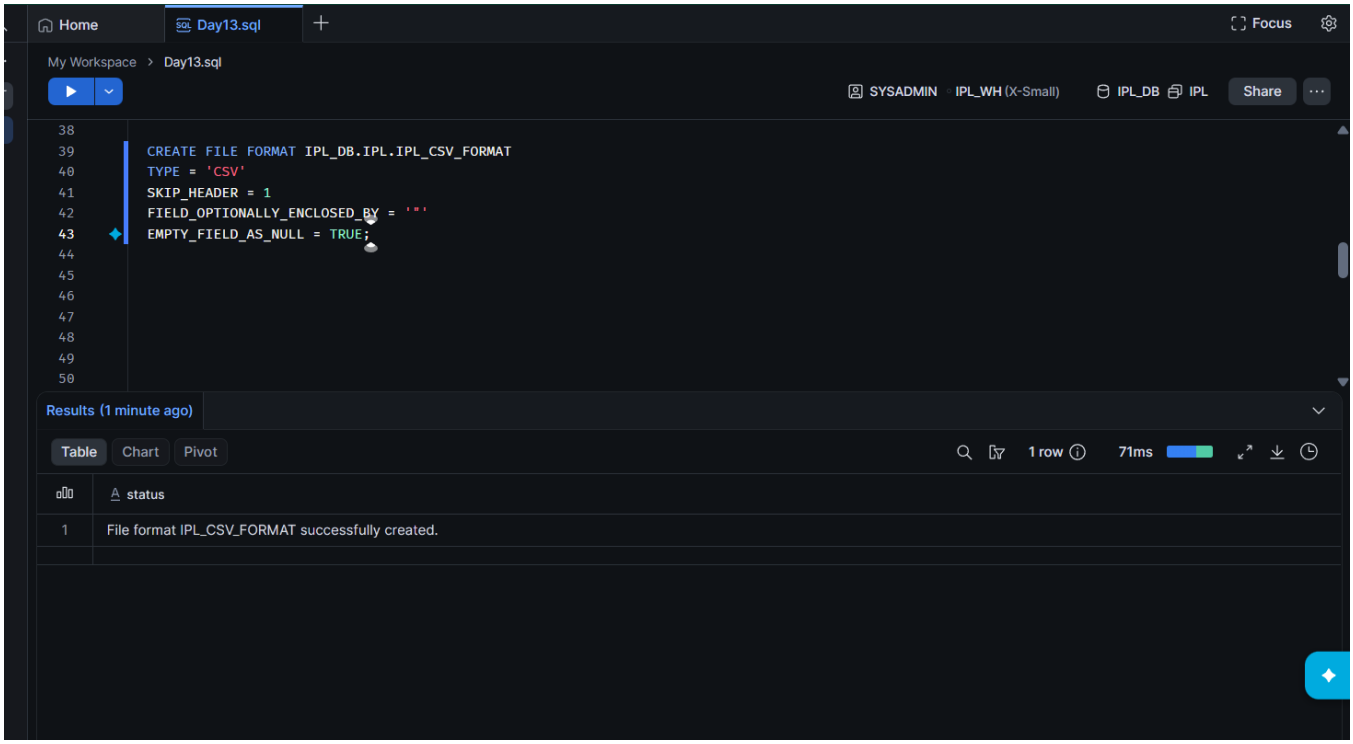
Step 12: Create CSV File Format

SOL Command

```
CREATE FILE FORMAT IPL_DB.IPL.IPL_CSV_FORMAT
TYPE = 'CSV'
SKIP_HEADER = 1
FIELD_OPTIONALLY_ENCLOSED_BY = '"'
EMPTY_FIELD_AS_NULL = TRUE;
```

Output

File format IPL_CSV_FORMAT successfully created.



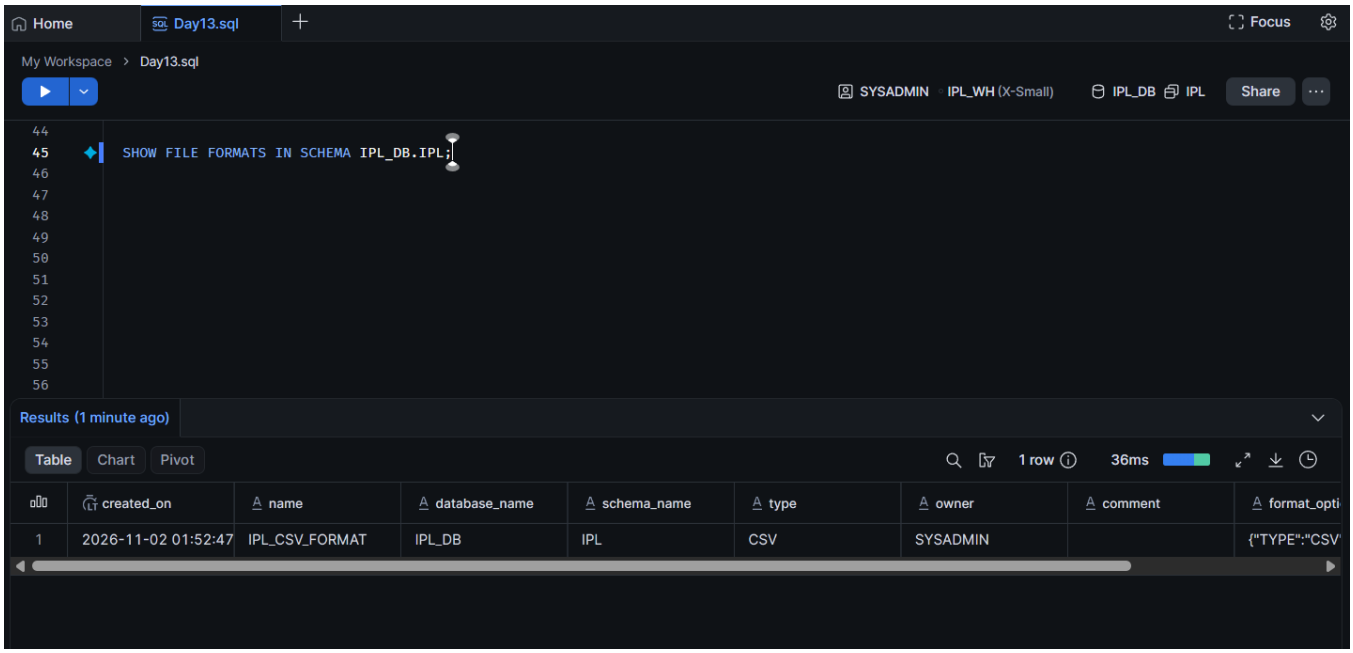
Step 13: Verify CSV File Format

SOL Command

```
SHOW FILE FORMATS IN SCHEMA IPL_DB.IPL;
```

Output

IPL_CSV_FORMAT is listed with type CSV.



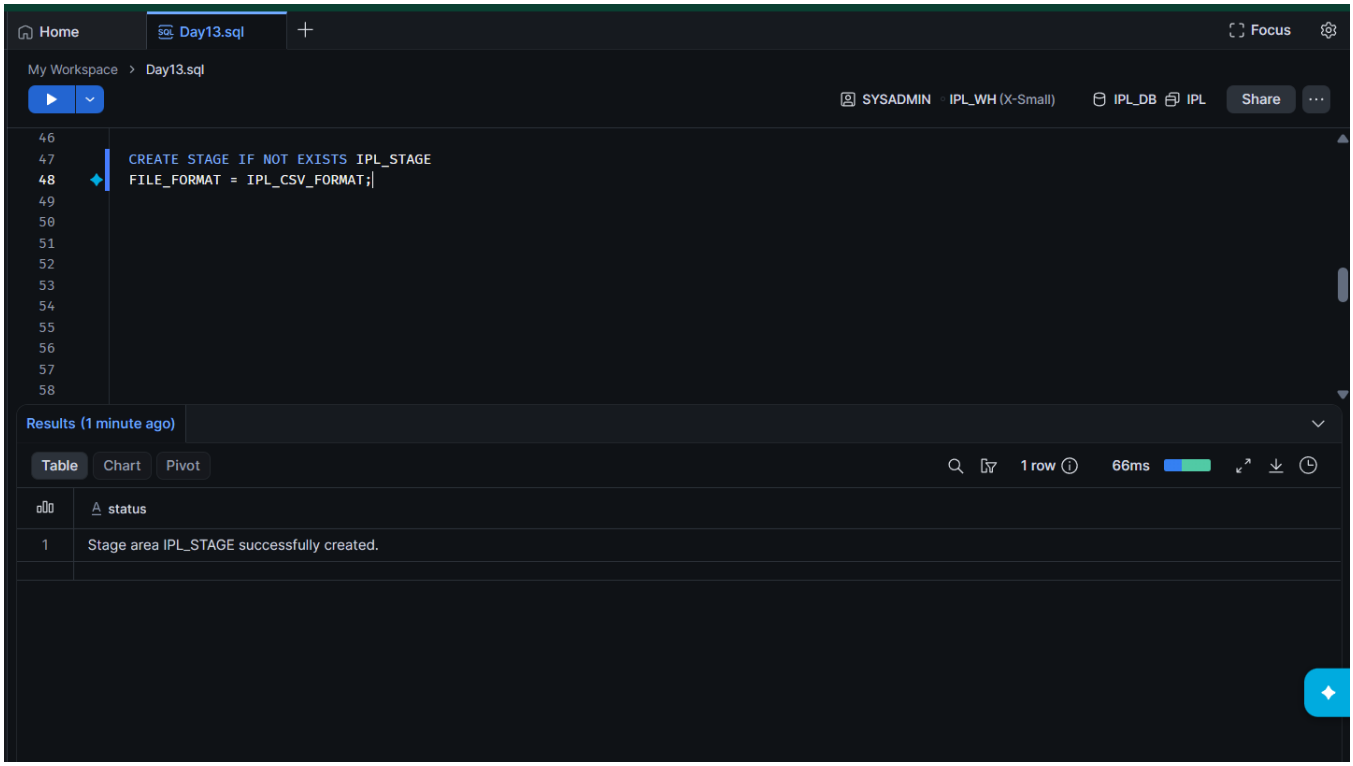
Step 14: Create Internal Stage

SOL Command

```
CREATE STAGE IF NOT EXISTS IPL_STAGE
FILE_FORMAT = IPL_CSV_FORMAT;
```

Output

Stage area IPL_STAGE successfully created.



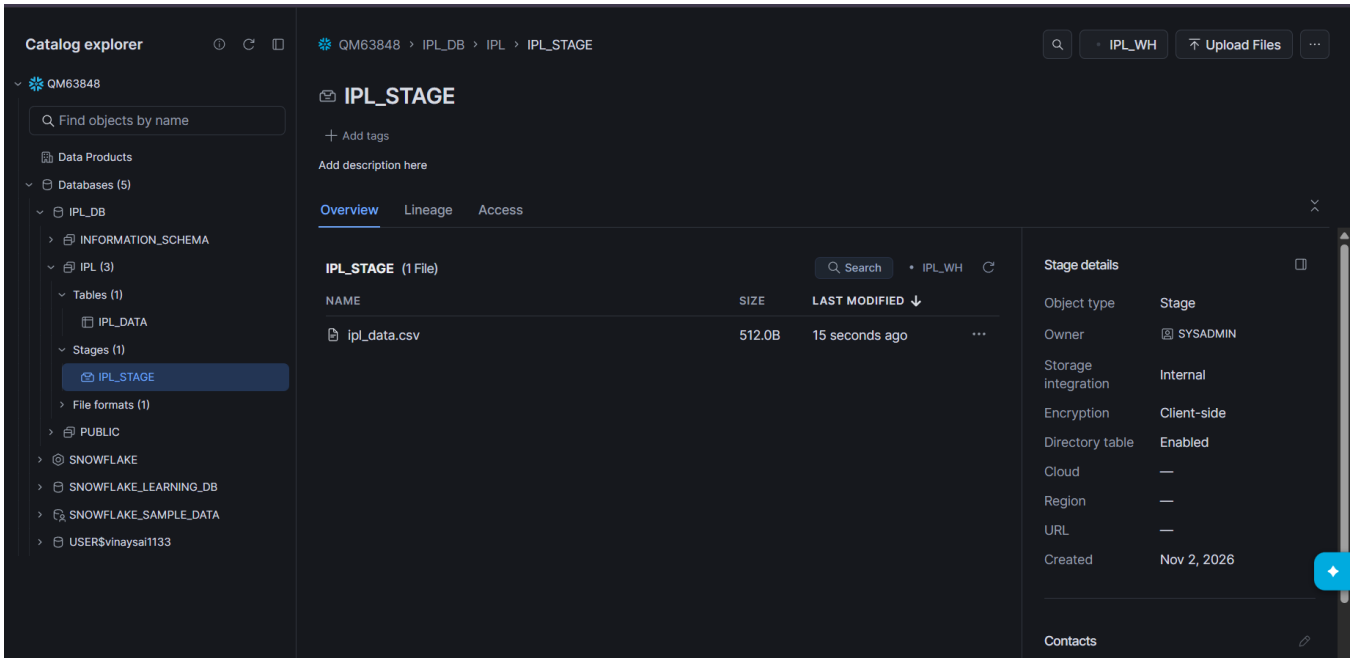
Step 15: Upload CSV File to Internal Stage

SOL Command

```
Snowflake UI: Catalog Explorer → IPL_DB → IPL → Stages → IPL_STAGE → Upload Files.
```

Output

The file ipl_data.csv is visible in IPL_STAGE.



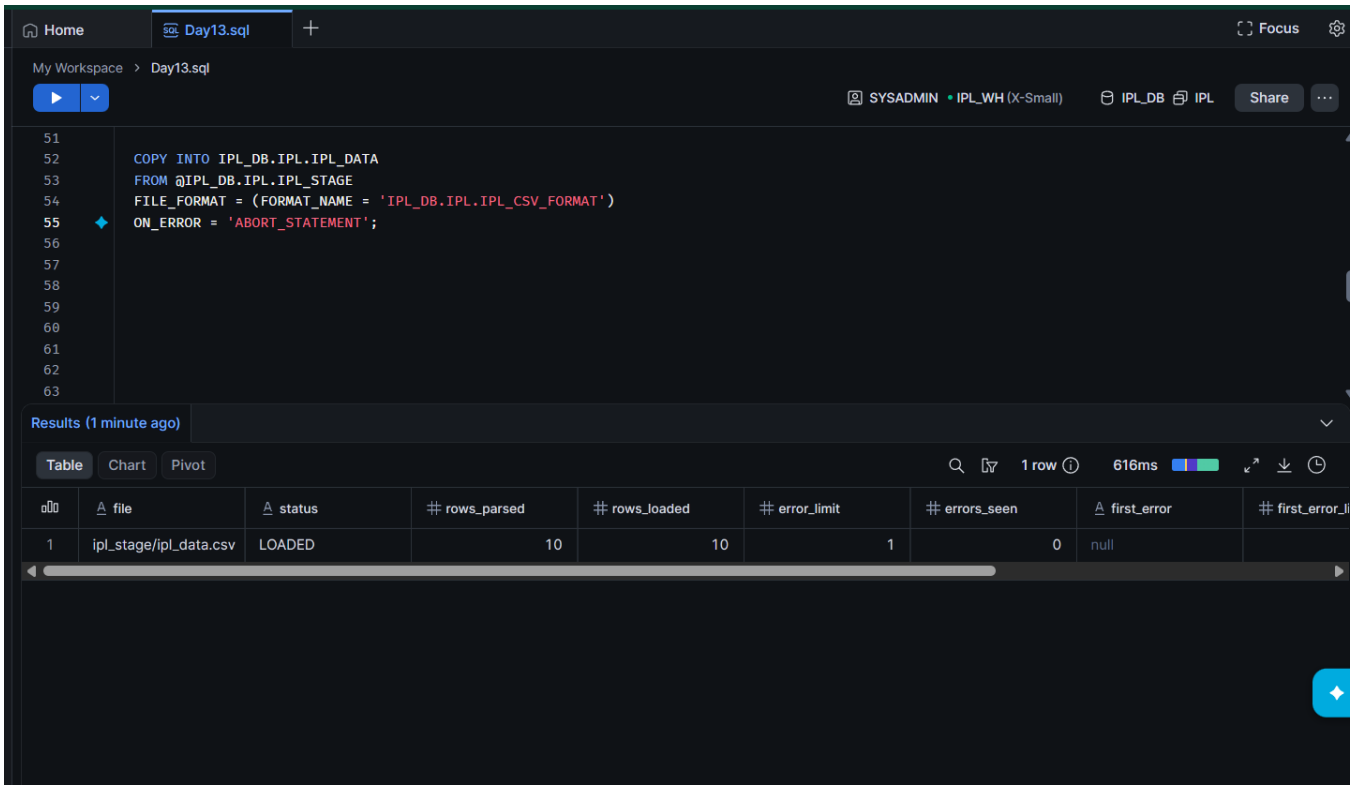
Step 16: Load CSV Data into IPL Table

SOL Command

```
COPY INTO IPL_DB.IPL.IPL_DATA
FROM @IPL_DB.IPL.IPL_STAGE
FILE_FORMAT = (FORMAT_NAME = 'IPL_DB.IPL.IPL_CSV_FORMAT')
ON_ERROR = 'ABORT_STATEMENT';
```

Output

File status: LOADED. 10 rows parsed, 10 rows loaded, 0 errors.



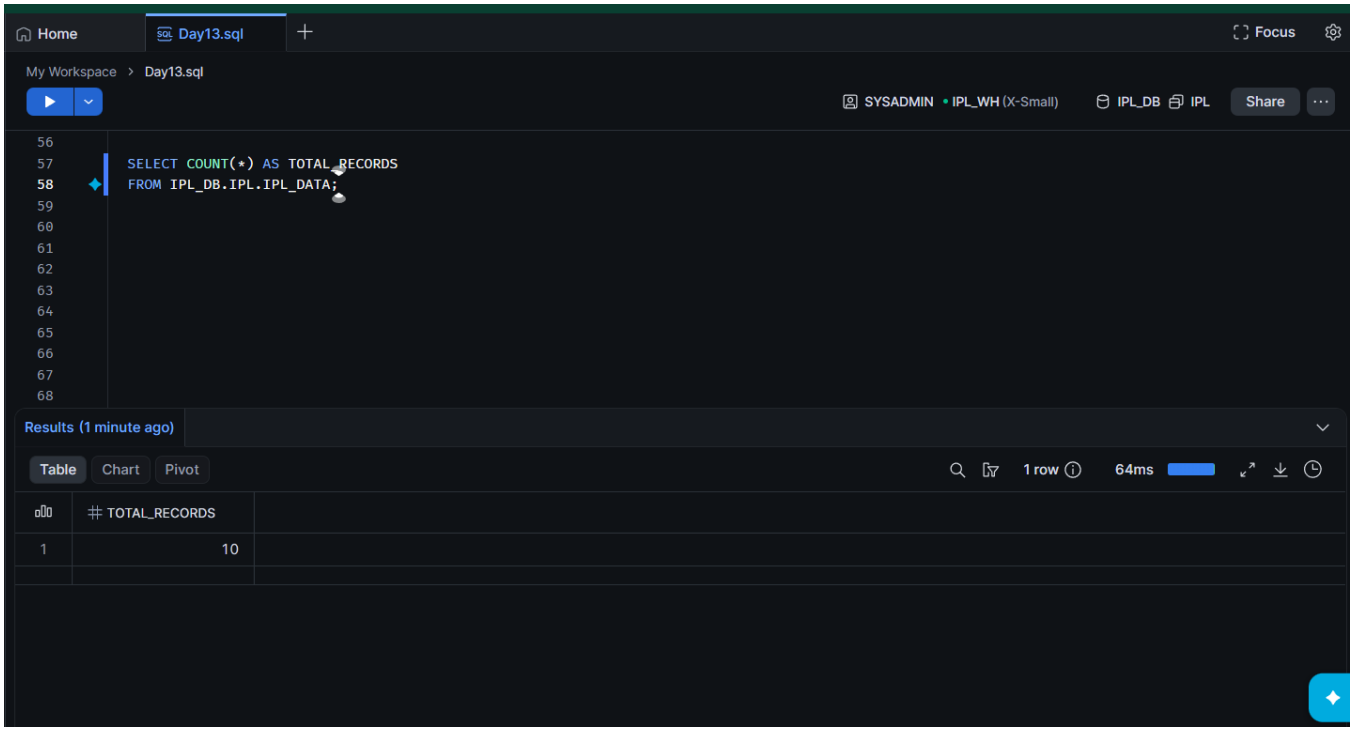
Step 17: Count Total Records

SOL Command

```
SELECT COUNT(*) AS TOTAL_RECORDS
FROM IPL_DB.IPL.IPL_DATA;
```

Output

The query returned 10, confirming that the table contains 10 records.



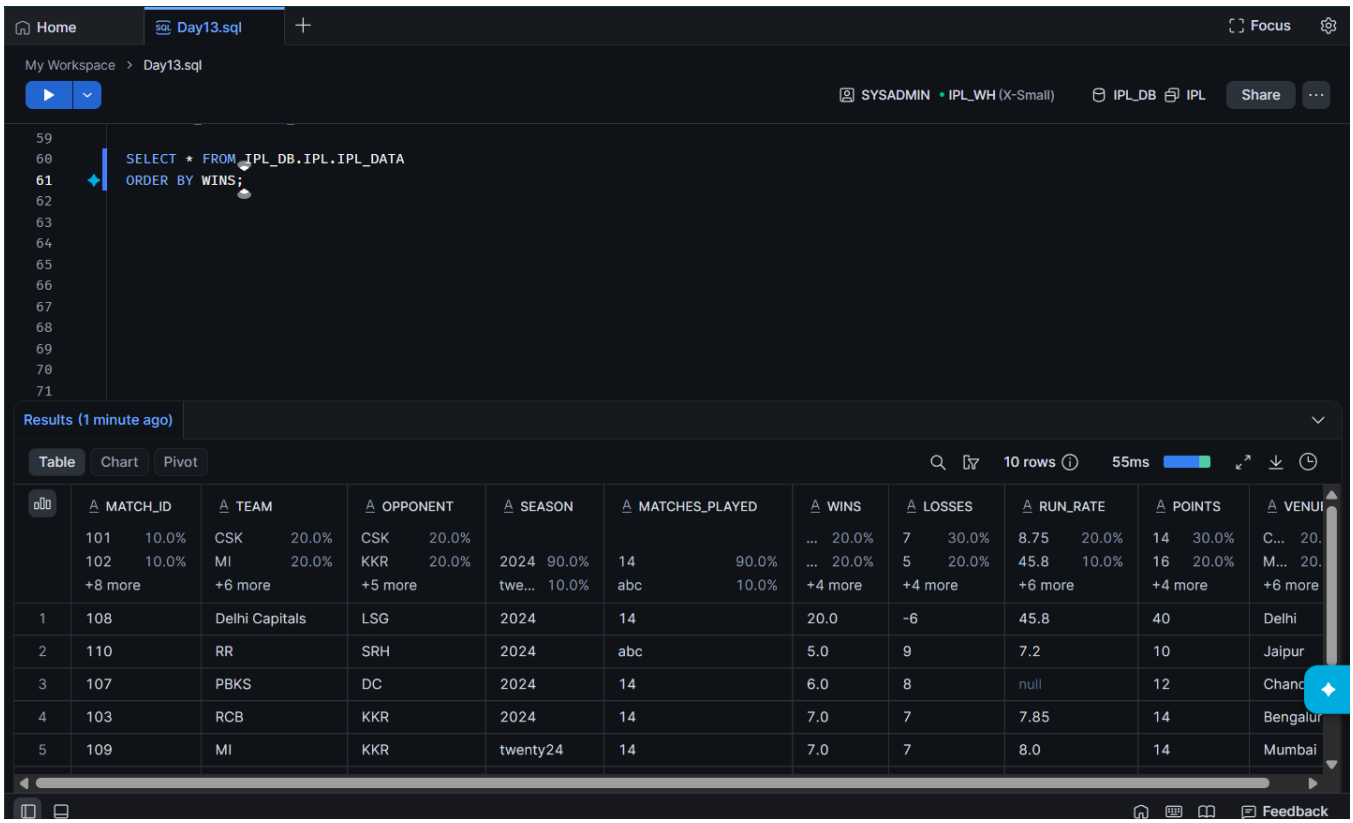
Step 18: View Records Ordered by Wins

SQL Command

```
SELECT * FROM IPL_DB.IPL.IPL_DATA
ORDER BY WINS;
```

Output

The records are displayed in ascending order of WINS by default.



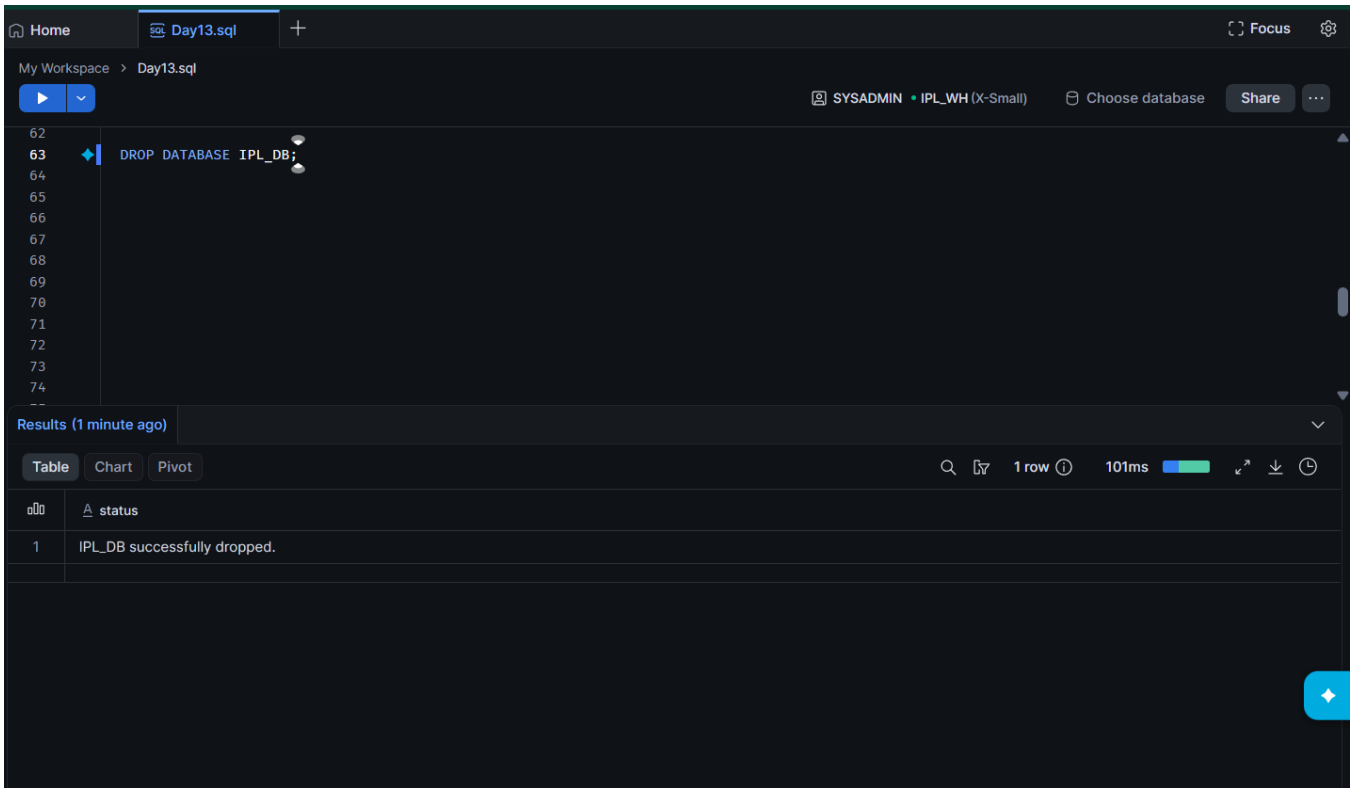
Step 19: Drop the IPL Database

SOL Command

```
DROP DATABASE IPL_DB;
```

Output

The result confirms that IPL_DB was successfully dropped.



Step 20: Drop the IPL Warehouse

SOL Command

```
DROP WAREHOUSE IF EXISTS IPL_WH;
```

Output

The result confirms that IPL_WH was successfully dropped.

